



Algoritmos e Programação

Revisão para Prova I

Prof. Pedro Bilar Montero
Universidade Federal de Santa Maria
Colégio Politécnico
Curso Técnico em Informática Para Internet

O que vamos revisar hoje?

Tipos de dados e operadores básicos

Entrada e Saída de Dados

Desvios Condicionais

Estruturas de Repetição

Linguagem C — Visão Geral

- ▶ Base para C++, Java, JavaScript, etc.
- ▶ **Compilada:** código → compilador → executável
- ▶ Mais controle = mais responsabilidade

Estrutura mínima

```
1 #include <stdio.h>
2
3 int main() {
4     // Seu código aqui
5     return 0;
6 }
```

- ▶ `#include <stdio.h>` — biblioteca de entrada/saída
- ▶ `int main()` — ponto de partida
- ▶ `return 0;` — fim com sucesso

Tipos de Dados Básicos em C

`int`

Números **inteiros**

Formato: `%d`

```
int idade = 18;
```

`float`

Números **decimais**

Formato: `%f`

```
float nota = 8.5;
```

`char`

Um único **caractere**

Formato: `%c`

```
char x = 'A';
```

Atenção

Em C, toda variável precisa ter um **tipo declarado** antes de ser usada!

Declarando e Inicializando Variáveis

Apenas declarar:

```
1 int idade;  
2 float raio;  
3 char letra;
```

Declarar e inicializar:

```
1 int idade = 16;  
2 float pi = 3.14;  
3 char letra = 'A';
```

Constantes com const

Valor fixo que **não pode ser alterado** durante o programa:

```
1 const float PI = 3.14;  
2 const int ANO = 2026;
```

Operadores Aritméticos e Precedência

Exemplos:

Op.	Nome	Prioridade
-	Inversão de sinal	0
*	Multiplicação	1
/	Divisão	1
%	Módulo (resto)	1
+	Adição	2
-	Subtração	2

```
1 // resultado = 20 (5*2 primeiro)
2 int r = 10 + 5 * 2;
3
4 // resultado = 30 (parênteses
5   primeiro)
6 int r = (10 + 5) * 2;
7
8 // resto da divisão
9 int resto = 17 % 5; // = 2
```

Regra

Use **parênteses** para garantir a ordem desejada nas operações!

Especificadores de Formato

O que são?

Especificadores (começam com %) indicam ao printf/scanf o **tipo** do valor que será exibido ou lido.

Espec.	Tipo
%d	int — inteiro
%f	float — decimal
%f	float
%c	char — caractere

Precisão do %f

Por padrão, %f mostra **6 casas decimais**. Para controlar a precisão, use %.Nf: por exemplo, %.2f mostra 2 casas e %.1f mostra 1 casa decimal.

printf() — Saída de Dados

Sintaxe

```
printf("texto %especificador", variável);
```

Espec.	Tipo
%d	int
%f	float
%c	char
\n	quebra de linha
\t	tabulação

```
1 int idade = 16;
2 float nota = 8.75;
3
4 printf("Idade: %d\n", idade);
5 // Saída: Idade: 16
6
7 printf("Nota: %.2f\n", nota);
8 // Saída: Nota: 8.75
```


scanf() — Entrada de Dados

Sintaxe

```
scanf("%especificador", &variável);
```

Uso correto:

```
1 int idade;  
2 float raio;  
3 char letra;  
4  
5 scanf("%d", &idade);  
6 scanf("%f", &raio);  
7 scanf(" %c", &letra);
```

Erros comuns:

```
1 // ERRADO: falta o &  
2 scanf("%d", idade);  
3  
4 // ERRADO: tipo errado  
5 // (idade eh int, nao float)  
6 scanf("%f", &idade);
```

Lendo caracteres!

Para ler um char com scanf, use um espaço antes do %c para evitar ler lixo da entrada:

Enunciado

Escreva um programa em C que leia a **inicial do nome do aluno** (char), sua **idade** (int) e sua **nota final** (float). O programa deve exibir na tela uma mensagem formatada como abaixo, com a nota exibida com **1 casa decimal**.

Dicas

Declare as variáveis com os tipos corretos. Use %c, %d e %.1f. Não esqueça do & no scanf()!

Qual será a saída exata na tela quando o usuário digitar 12 para a e 5 para b?

```
#include <stdio.h>
int main() {
    int a, b;
    int soma, diferenca, resto;

    scanf("%d", &a);
    scanf("%d", &b);

    soma      = a + b * 2;
    diferenca = a - b;
    resto     = a % b;

    printf("Resultado 1: %d\n", soma);
    printf("Resultado 2: %d\n", diferenca);
    printf("Resultado 3: %d\n", resto);

    return 0;
}
```

Exercício — Gabarito e Discussão

Com $a = 12$ e $b = 5$:

Cálculos passo a passo

- ▶ $\text{soma} = 12 + 5 * 2$
 $\Rightarrow 5 * 2 = 10$ (precedência!)
 $\Rightarrow 12 + 10 = 22$
- ▶ $\text{diferenca} = 12 - 5 = 7$
- ▶ $\text{resto} = 12 \% 5 = 2$
 $(12 = 2*5 + 2)$

Saída na tela

Resultado 1: 22
Resultado 2: 7
Resultado 3: 2

Ponto de atenção

$\text{soma} = a + b * 2$ **não** é $(a+b)*2$! A multiplicação tem precedência maior que a adição.

Desvios Condicionais — if, else if e else

O que fazem?

Permitem que o programa escolha caminhos diferentes com base em uma **condição**.

Sintaxe básica

```
if (condicao) {  
    // executa se for verdadeiro  
} else if (outra_condicao) {  
    // executa em outro caso  
} else {  
    // executa se nada acima ocorrer  
}
```

Lembrete

Em C, a condição é considerada **verdadeira** quando é diferente de zero. O valor **0** representa **falso**.

Exemplo mental

idade $\geq$ 18 decide se a pessoa é maior de idade.



Operadores Relacionais

Op.	Significado	Exemplo
==	Igual a	x == y
!=	Diferente de	x != y
>	Maior que	x > y
<	Menor que	x < y
>=	Maior ou igual	x >= y
<=	Menor ou igual	x <= y

Atenção

Use == para comparação. O símbolo = serve para **atribuição**.

Operadores Lógicos

Op.	Significado
&&	E lógico
	OU lógico
!	Não lógico

Quando usar

- ▶ && exige que **as duas** condições sejam verdadeiras
- ▶ || aceita **uma ou outra** condição verdadeira
- ▶ ! inverte o resultado lógico

Enunciado

Escreva um programa em C que leia a **nota final** de um aluno e informe sua situação. Se a nota for maior ou igual a 7, exiba Aprovado. Se for maior ou igual a 5, exiba Recuperacao. Caso contrário, exiba Reprovado.

Dicas

Use `if / else if / else` e lembre que a ordem das condições importa.

Qual será a saída quando o usuário digitar 17 para idade e 1 para autorizacao?

```
#include <stdio.h>

int main() {
    int idade, autorizacao;

    scanf("%d", &idade);
    scanf("%d", &autorizacao);

    if (idade >= 18 || autorizacao == 1) {
        printf("Pode entrar\n");
    } else {
        printf("Nao pode entrar\n");
    }

    if (idade >= 18 && autorizacao == 1) {
        printf("Entrada livre\n");
    }

    return 0;
}
```

Exercício — Gabarito e Discussão

Com `idade = 17` e `autorizacao = 1`:

Avaliação lógica

- ▶ `idade >= 18` é falso
- ▶ `autorizacao == 1` é verdadeiro
- ▶ `falso || verdadeiro` resulta em verdadeiro
- ▶ `falso && verdadeiro` resulta em falso

Saída na tela

Pode entrar

Ponto de atenção

O `&&` só será verdadeiro se **as duas** condições forem verdadeiras.

Laços de Repetição — while, do while e for

Para que servem?

Repetem um bloco de código enquanto uma condição for verdadeira.

while

Testa antes de executar.

```
while (i < 5) {  
    i++;  
}
```

do while

Executa ao menos uma vez.

```
do {  
    i++;  
} while (i < 5);
```

for

Ideal com contador.

```
for (i = 0; i < 5; i++) {  
    // repeticao  
}
```

Contador e Acumulador

Contador

Variável que **conta** quantas vezes algo acontece.

```
int contador = 0;
for (int i = 0; i < 5; i++) {
    contador++;
}
```

Acumulador

Variável que **acumula** valores ao longo do laço (soma, produto, etc.).

```
int soma = 0;
for (int i = 1; i <= 5; i++) {
    soma = soma + i;
}
```

Atenção

Sempre inicialize contador/acumulador antes do laço para evitar resultados incorretos.

Enunciado

Escreva um programa em C que leia um número inteiro n e calcule a soma de todos os números de 1 até n usando um laço `for`. Ao final, exiba o resultado.

Dicas

Use um acumulador iniciado em 0. A cada repetição, some o valor atual de i .

Enunciado

Analise o código abaixo e responda: **qual será a saída exata na tela** quando $n = 4$?

```
#include <stdio.h>

int main() {
    int n = 4;
    int i;
    int produto = 1;

    for (i = 1; i <= n; i++) {
        produto = produto * i;
        printf("i=%d produto=%d\n", i, produto);
    }

    return 0;
}
```

Exercício — Gabarito e Discussão

Com $n = 4$:

Passo a passo

- ▶ Início: produto = 1
- ▶ $i = 1$: produto = 1
- ▶ $i = 2$: produto = 2
- ▶ $i = 3$: produto = 6
- ▶ $i = 4$: produto = 24

Saída na tela

```
i=1 produto=1  
i=2 produto=2  
i=3 produto=6  
i=4 produto=24
```

Ponto de atenção

O valor de produto depende do resultado da iteração anterior (acumulador multiplicativo).

Bons estudos e boa prova!